

Expense & Budget Visualizer

Preparations and Submission Details

- **The deadline** will be on **Saturday** at 11.59 (WIB)
 - Link to submit the assignment [here](#) (will be opened on **Wednesday**)
 - Please follow the repository naming format below:
 - * **CodingCamp-[batch date (ddmmyy)]-[participantname]**
 - * *Example: **CodingCamp-3August26-yamaroni***
 - * *Note: The batch date is the **first day you take this course** (3 August 2026), not the day you finish the assignment.*
-

Introduction

In this 5-day course, you will learn the basics of **software engineering** by building a small web application from start to finish.

Before You Start

1. **Sign up for [Kiro](#)** (and make sure you remember your **Builder ID** as you need it to submit it on the project submission form later)

Kiro is the **main tool** you will use to help you:

- Understand the task
- Break features into smaller steps
- Get help with code and debugging

2. **Other Tools Needed**

Make sure you also have:

- GitHub Desktop
 - A GitHub account
-

Required Features (MVP)

Your project **must include**:

- **Input Form**
 - Fields: **Item Name, Amount, Category** (Food, Transport, Fun)
 - Add a transaction to the list when submitted
 - Validate that all fields are filled

- **Transaction List**
 - Scrollable list of all added items
 - Show name, amount, and category
 - Ability to delete items
 - **Total Balance**
 - Display total balance at the top
 - Updates automatically when items are added or deleted
 - **Visual Chart**
 - Pie chart showing spending distribution by category
 - Chart updates automatically when items change
 - You can use **Chart.js** or another simple chart library
-

Project Overview

You will build an **Expense & Budget Visualizer**. This is a mobile-friendly web app that helps users track their daily spending. It shows their total balance, a history of transactions, and a visual chart of spending by category. You can build it as a simple web page that works well on phones.

Constraint *(Please copy this on Kiro):*

Technical Constraints

TC-1: Technology Stack

- HTML for structure
- CSS for styling
- Vanilla JavaScript (no frameworks like React, Vue, etc.)
- No backend server required

TC-2: Data Storage

- Use browser Local Storage API
- All data stored client-side only

TC-3: Browser Compatibility

- Must work in modern browsers (Chrome, Firefox, Edge, Safari)
- Can be used as standalone web app or browser extension

Non-Functional Requirements

NFR-1: Simplicity

- Clean, minimal interface
- Easy to understand and use
- No complex setup required
- No test setup required

NFR-2: Performance

- Fast load time
- Responsive UI interactions
- No noticeable lag when updating data

NFR-3: Visual Design

- User-friendly aesthetic
- Clear visual hierarchy
- Readable typography

Folder Rules

- Only **1 CSS file** inside `css/`
- Only **1 JavaScript file** inside `js/`
- Keep code clean and readable

GitHub & Deployment *(Only basic Git is needed)*

- Use GitHub Desktop to save your work
- Push your code to GitHub
- Publish the site using GitHub Pages

Optional Challenges *(Choose 3)*

Choose 3 out of these 5 options to make your project more interactive and user-friendly:

- ☐ Allow users to add custom categories
- ☐ Add a monthly summary view
- ☐ Sort transactions by amount or category
- ☐ Highlight spending over a set limit
- ☐ Dark/light mode toggle

Submission

1. After finish your website, upload the Source Code on **your Github Repository** and publish the website in [Github Pages](#)

2. Submit your **AWS Builder ID** (make sure in your project the **.kiro** folder is present), **Github Repo URL** and **published website** through dedicated [Paperform submission](#) that will be opened on **Thursday**

Important: If you do not submit **all required links** on Paperform, your submission will **not be valid**.

Example of expected Output:

